

AZURE DATA ENGINEERING

Detailed Learning Notes — Chapters 1 to 3

Based on the supplied course transcript • Beginner-friendly • Interview-focused • Practical

How to use these notes — Read the Simple Explanation first, then study the diagram/table, reproduce the practical example, and finish with the interview + revision sections. “Expanded explanation” content is added to make the course concepts easier to learn; it is not presented as a verbatim statement from the instructor.

Chapter	Coverage
1. Azure & Data Fundamentals	Cloud, service models, resilience, Azure hierarchy, networking, data types, Azure SQL/Cosmos DB, Storage/ADLS
2. ADF Core Fundamentals	ETL/ELT, connectors, linked services, datasets, pipelines, Copy Activity, Studio, first pipeline
3. Advanced ADF & Production Orchestration	Metadata-driven pipelines, ForEach/If, parameters, data flows, triggers, Execute Pipeline, production flow

Chapter 1 — Azure & Data Fundamentals

Course coverage: approximately 00:13:51 → 02:18:44. The instructor builds the foundation from cloud computing through Azure data-storage services before transitioning into Azure Data Factory.

1.1 What is cloud computing?

Simple explanation: Cloud computing means renting computing resources—such as servers, storage, databases and networking—from a provider over the internet instead of buying and operating all of the hardware yourself.

Course concept — The course first introduces cloud through remote data storage and then expands the idea to on-demand computing resources and pay-as-you-go usage.

Cloud computing: resources delivered on demand



Figure 1. Cloud computing in one picture

Why companies use it: demand changes. A startup may need modest capacity today and much more during a sale or product launch. Cloud makes it possible to provision and release resources without purchasing every physical machine in advance.

Concept	Plain-English meaning	Data-engineering relevance
On-demand	Create resources when needed	Spin up compute/storage for pipelines and workloads
Pay-as-you-go	Pay based on consumption	Costs follow actual usage instead of fixed hardware purchase
Elasticity	Increase/decrease capacity	Handle variable ingestion or transformation workloads
Remote access	Use services through network endpoints	Pipelines connect to cloud sources/sinks from many locations

Interview: What is cloud computing?

A delivery model for computing resources over a network on demand, typically with elastic capacity and usage-based pricing.

1.2 Data centers, servers and endpoints

A cloud service is still backed by physical hardware. Providers operate data centers containing servers, storage and networking. Azure exposes logical resources—such as a VM or database—while hiding most physical infrastructure details.

More precise understanding — A cloud is not simply “a place to dump data.” Storage is one cloud capability. Cloud computing also includes compute, networking, databases, analytics, identity and managed application services.

1.3 On-premises vs IaaS vs PaaS vs SaaS

Cloud service models: control vs managed responsibility



Figure 2. Moving from on-premises toward more provider-managed services

Model	You manage most of	Provider manages most of	Course example
On-premises	Hardware, OS, network, apps, data	Nothing in your environment	Own servers
IaaS	OS, applications, data, many configurations	Physical host/network/data center	Azure Virtual Machine
PaaS	Data, app logic, selected configuration	Infrastructure + OS/platform	Azure SQL Database
SaaS	Usage, identities, your data	Application + platform + infrastructure	Microsoft Fabric-style SaaS

Memory trick — IaaS = rent the machine. PaaS = rent a ready platform. SaaS = use the finished software.

Interview: Why would a data team prefer PaaS over IaaS for a database?

PaaS reduces operational work such as OS/platform maintenance so the team can focus more on data, schema, security configuration and workload design.

Common mistakes

- Thinking IaaS means Microsoft manages your guest OS and applications.
- Thinking SaaS removes all customer responsibility; identities, permissions and data governance still matter.

- Treating Azure and “cloud” as exact synonyms—Azure is one cloud provider/platform.

1.4 Fault tolerance, availability zones and regions

The course motivates resilience by asking what happens when a server, data center, city/zone or region fails. The core idea is redundancy: keep additional copies or deploy resources across failure boundaries so one failure does not automatically destroy the service.

Failure boundary	Azure idea	Purpose
Single hardware/server	Redundancy/replication	Survive component failure
Data-center/zone failure	Availability Zones	Separate physical locations within a region
Regional disaster	Geo-redundancy / paired-region strategies	Maintain a copy in another geographic region

Important — Replication does not automatically mean zero downtime or zero data loss. The exact behavior depends on the Azure service and redundancy option you configure.

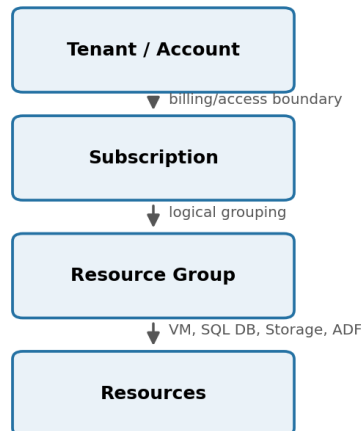
1.5 Storage redundancy: LRS, ZRS, GRS and GZRS

Option	Replication idea	Use when
LRS	Multiple copies in one local data-center scope	Lowest-cost local durability is sufficient
ZRS	Copies across availability zones in one region	Need resilience to a zone outage
GRS	Local copies + asynchronous copy to secondary region	Need regional disaster protection
GZRS	Zone redundancy in primary + geo copy	Need both zone and regional resilience

Cost principle — More redundancy usually means higher cost. Choose it from business RPO/RTO and durability requirements—not simply “maximum is always best.”

1.6 Azure hierarchy: subscription, resource group and resources

Simplified Azure resource hierarchy



A resource is an Azure service instance you create, such as a Storage Account, Azure SQL Database, VM or Data Factory. A resource group is a logical container used to organize related resources. A subscription provides a billing/access boundary for resources.

Term	Think of it as	Example
Subscription	Billing + access boundary	Training subscription
Resource Group	Project folder	rg-data-engineering-dev
Resource	Actual service instance	adf-sales-dev, storage account, SQL DB

1.7 Networking: public and private access

The transcript introduces the idea that an Azure resource can be exposed through a public endpoint or restricted through private networking. For learning labs, public access is convenient; for sensitive production data, private connectivity and strict firewall/network rules are common.

Data-engineering connection — ADF, Databricks, SQL databases and Storage Accounts often need to communicate across network boundaries. A pipeline can be logically correct but still fail because DNS, firewall or private-endpoint rules block the connection.

1.8 Types of data

Data types and common Azure storage choices

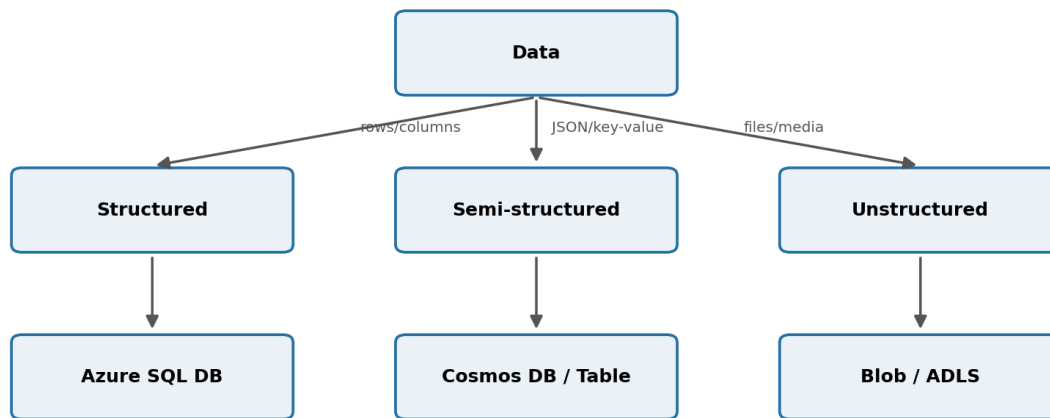


Figure 4. Data types and common Azure services

Type	Shape	Examples	Typical course service
Structured	Fixed rows/columns/schema	Customer table, orders	Azure SQL Database
Semi-structured	Self-describing, flexible fields	JSON, key-value documents	Cosmos DB / Table Storage
Unstructured	No fixed tabular schema	Images, video, PDFs	Blob Storage / ADLS

```
-- Structured-data example
CREATE TABLE Orders (
```

```

    order_id INT,
    customer_id INT,
    amount DECIMAL(12,2),
    order_date DATE
);

SELECT customer_id, SUM(amount) AS revenue
FROM Orders
GROUP BY customer_id;

# PySpark can read semi-structured JSON and infer a schema
df = spark.read.json("/raw/customers/*.json")
df.printSchema()
df.select("id", "name").show()

```

1.9 Azure SQL Database and relational data

Azure SQL Database is presented as the managed relational option for structured data. Relational systems use tables, schemas, keys and SQL. For a data engineer, operational databases are commonly sources from which data is extracted into a lake or warehouse.

ETL mindset — An application database is optimized for application transactions. Analytics commonly moves or replicates that data into analytical storage rather than running every BI workload directly against the operational database.

1.10 Cosmos DB and NoSQL

Cosmos DB is introduced for non-relational/semi-structured scenarios. The course walks through APIs such as NoSQL, MongoDB, Table, Gremlin and Cassandra and explains that different APIs fit different data models or compatibility needs.

API/model	Simple mental model	Example use
Cosmos DB for NoSQL	JSON/document model with SQL-like query language	Flexible application documents
MongoDB compatibility	Document model for MongoDB-oriented applications	Existing MongoDB ecosystem
Table	Key/attribute style storage	Simple key-value entities
Gremlin	Graph: nodes + relationships	Networks, relationships
Cassandra	Wide-column / column-family model	Large distributed wide-column workloads

```

-- Illustrative Cosmos DB for NoSQL query style
SELECT c.id, c.name
FROM c
WHERE c.country = "India"

```

The transcript also introduces HTAP (hybrid transactional/analytical processing) as a way to think about operational data becoming available for analytics. For a data engineer, Cosmos DB is often a source feeding downstream analytical systems.

Interview: Relational vs NoSQL—what is the key distinction?

Relational databases organize data into schema-governed tables and relationships. NoSQL systems support alternative models such as documents, key-value, graph or wide-column structures and often allow more flexible schemas.

1.11 Azure Storage Account

A Storage Account is the umbrella resource through which the course explores Blob/containers, Data Lake, file shares, queues and tables.

ADLS Gen2 hierarchy with hierarchical namespace

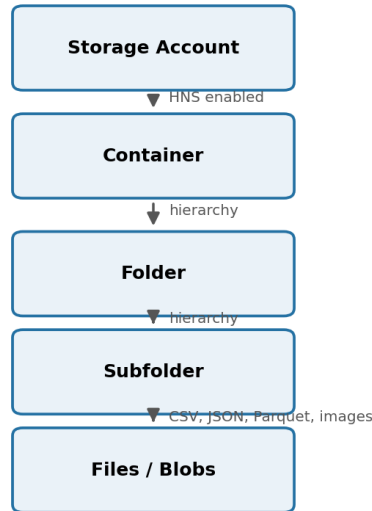


Figure 5. ADLS Gen2 hierarchical organization

Storage capability	What it stores/does	Typical DE use
Blob / Containers	Objects/files	Raw files, logs, exports
ADLS Gen2	Blob storage + hierarchical namespace	Data-lake folder hierarchy and analytics
Azure Files	Managed file shares	Shared file-system scenarios
Queue Storage	Messages	Simple asynchronous messaging
Table Storage	Key/attribute entities	Simple semi-structured entities

1.12 ADLS Gen2 and hierarchical namespace

The course shows that enabling hierarchical namespace gives Data Lake Storage Gen2 directory semantics. Instead of thinking only in flat object names, you can organize data as containers → folders → subfolders → files.

```

raw/
  sales/
    2026/
      09/
        22/
          sales_20260922.csv

curated/
  sales/
    daily_sales.parquet
  
```

Real-world DE — A common lake layout separates raw/bronze data from cleaned/silver and business/gold data. The exact naming convention varies by organization.

1.13 Hot, Cool, Cold/Archive-style access tiers

Tier idea	Access pattern	Trade-off
-----------	----------------	-----------

Hot	Frequently accessed	Higher storage cost, lower access cost
Cool	Infrequently accessed	Lower storage cost, higher retrieval/access considerations
Cold / archive-style	Rarely accessed/retention	Lowest storage economics but slower/costlier retrieval depending on tier

More precise understanding — Azure storage tier names and minimum-retention/retrieval behavior can evolve. Treat the course explanation as the conceptual model and check current Azure documentation before choosing a production tier.

Quick revision

Remember	Meaning
Cloud	On-demand compute/storage/network/database services
IaaS/PaaS/SaaS	Increasing provider management
Resource Group	Logical container for Azure resources
Structured	Rows + columns + schema
Semi-structured	JSON/key-value/flexible fields
ADLS Gen2	Storage with hierarchical namespace for analytics
Cosmos DB	Managed NoSQL/multi-model database family

Chapter 2 — Azure Data Factory Core Fundamentals

Course coverage begins at approximately 02:18:57. The instructor moves from “what is ADF?” into connectors, linked services, datasets, activities, Studio and the first working copy pipeline.

2.1 What is Azure Data Factory?

Simple explanation: Azure Data Factory (ADF) is a cloud data-integration and orchestration service. It connects to sources and destinations, moves data, coordinates steps, and can invoke transformation services. The course also demonstrates Mapping Data Flows for visual transformations.

Azure Data Factory: source-to-sink orchestration

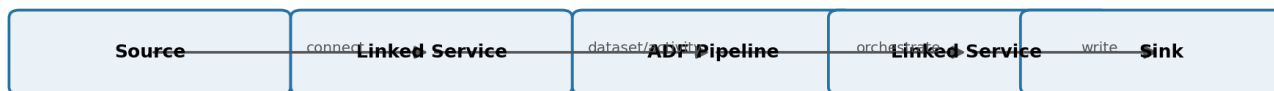


Figure 6. ADF connects sources to sinks and orchestrates work

Key idea — ADF is primarily the orchestrator/transport layer. It is not the place where your business data “lives”; the data normally lives in systems such as SQL databases, APIs, ADLS or other stores.

2.2 ETL vs ELT

Pattern	Order	Good mental model
ETL	Extract → Transform → Load	Transform before final analytical destination
ELT	Extract → Load → Transform	Land raw data first, transform using scalable analytical compute

```

# ELT-style PySpark transformation after raw data lands in the lake
bronze = spark.read.format("parquet").load("/bronze/orders")
silver = (bronze
    .dropDuplicates(["order_id"])
    .filter("amount >= 0"))
silver.write.mode("overwrite").parquet("/silver/orders")
  
```

2.3 Connectors, Linked Services and Datasets

These three terms are easy to mix up, so learn them as layers: a connector is the supported technology type; a Linked Service contains connection information; a Dataset points to the specific data structure/location you want an activity to read or write.

Object	Question it answers	Example
Connector	What technology can ADF talk to?	Azure SQL, ADLS, HTTP
Linked Service	How do I connect to that system?	Server/account + authentication
Dataset	Which data inside that system?	/raw/orders/orders.csv or dbo.Customers

Memory trick — Linked Service = connection. Dataset = data location/shape. Pipeline = workflow. Activity = action.

Interview: Linked Service vs Dataset?

A Linked Service defines connection information to a data store or compute service; a Dataset represents the specific data structure/location consumed or produced by an activity.

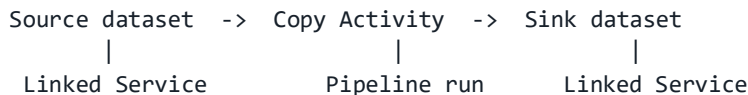
2.4 Pipeline, Activity, Source and Sink

A pipeline is the workflow container. Activities are the individual steps. In Copy Activity, the input side is the source and ADF commonly calls the destination the sink.

Term	Meaning
Pipeline	Workflow containing one or more activities
Activity	A unit of work: copy, metadata lookup, condition, data flow, etc.
Source	Where data is read from
Sink	Where data is written to

2.5 Copy Activity

Copy Activity is the workhorse for moving data between supported stores. The course builds a first pipeline that copies a file into another ADLS location and validates the result.



Performance idea — Copy Activity moves data; it does not automatically make the data analytically “clean.” Transformation is a separate concern unless you explicitly configure mappings or transformations.

2.6 ADF Studio workflow

The course demonstrates creating ADF, launching Studio, authoring pipelines, configuring linked services/datasets, debugging/running, monitoring, and publishing changes.

ADF area	Purpose
Author	Build pipelines, datasets, data flows
Monitor	Inspect pipeline/activity/trigger runs
Manage	Linked services, integration runtimes, triggers and related configuration
Publish All	Publish authored changes to the live factory

2.7 First practical pipeline

Scenario: create storage locations, connect ADF to ADLS, create source and sink datasets, drag Copy Activity into a pipeline, run it, and verify that the file appears at the destination.

Practice task — Rebuild the pipeline without following the video. If you can explain why you need two datasets and at least one connection, you understand the foundation.

Quick revision

Remember	Meaning
ADF	Integration + orchestration service
Linked Service	Connection information
Dataset	Specific data/location used by an activity
Pipeline	Workflow
Activity	One step inside a workflow
Sink	ADF term for destination
Copy Activity	Moves data between source and sink

Chapter 3 — Advanced ADF & Production Orchestration

This chapter organizes the advanced material from the supplied ADF transcript: metadata-driven processing, loops and conditions, parameterized datasets, transformations, triggers, pipeline composition, debugging and the final production-style workflow.

3.1 Why metadata-driven pipelines?

Hard-coding one filename works for a demo but breaks down when new files arrive. Metadata-driven design asks ADF to discover files/folders at runtime and then decide what to do with each item.

Metadata-driven selective file processing

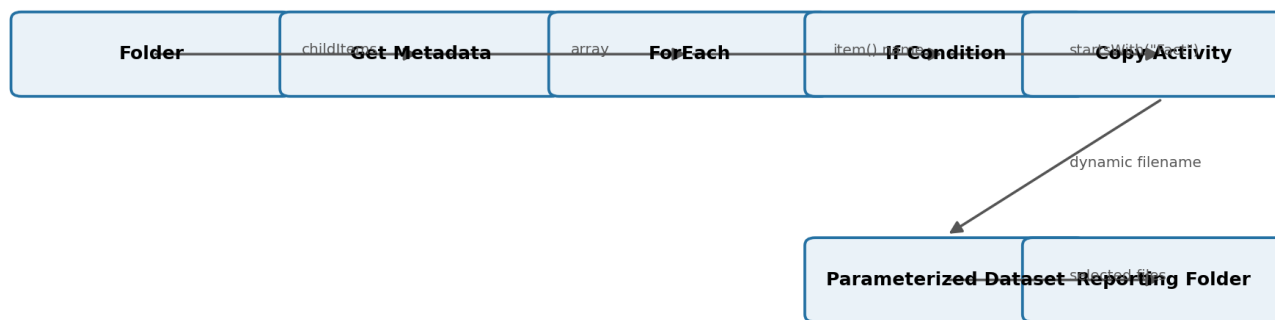


Figure 7. Metadata-driven selective file pipeline

3.2 Get Metadata Activity

Get Metadata reads information about a file/folder rather than copying its contents. In the course, the important output is childItems: an array describing items inside a folder.

Illustrative output shape

```
{
  "childItems": [
    {"name": "FactSales.csv", "type": "File"},
    {"name": "Customers.csv", "type": "File"}
  ]
}
```

Programming connection — childItems is an array. ForEach iterates through it. Each element is object/dictionary-like, so you access fields such as name and type.

3.3 ForEach Activity

ForEach is ADF's loop activity. Give it an array and it runs its inner activities for each element. The course discusses sequential execution and uses each current item in downstream expressions.

```
@activity('Get Metadata').output.childItems
```

```
# Inside the loop, conceptually:
@item().name
```

PySpark analogy — ForEach is orchestration-level looping. In Spark transformations, avoid Python row-by-row loops for big data; Spark DataFrame operations are distributed and are usually the right tool for data transformation.

3.4 If Condition and expression functions

The course filters files by name: if the current filename starts with “Fact”, execute the true branch and copy it; otherwise do nothing.

```
@startsWith(item().name, 'Fact')
```

This is a powerful pattern because the logic depends on metadata rather than a fixed list of files.

Interview: How would you copy only files whose names start with Fact?

Use Get Metadata to obtain childItems, ForEach over the array, If Condition with startsWith(item().name, 'Fact'), and Copy Activity in the true branch. Parameterize the dataset filename so each matching file is read dynamically.

3.5 Parameterized datasets

A parameter turns a hard-coded property into a runtime input. The course parameterizes the source filename so the same dataset definition can read different files during each ForEach iteration.

```
Dataset parameter: fileName
File path / name: @dataset().fileName
```

```
From ForEach/Copy Activity:
fileName = @item().name
```

Hard-coded dataset	Parameterized dataset
Works for one known filename	Reusable for many filenames
Requires edits for each file	Runtime value supplied by pipeline/activity
Poor for metadata-driven ingestion	Good for scalable ingestion patterns

3.6 Mapping Data Flow

The transcript introduces ADF Data Flow as the visual transformation capability backed by Spark. You configure transformations in a GUI while ADF runs distributed compute behind the scenes.

Transformation idea	What it does	SQL/PySpark equivalent
Filter	Keep matching rows	WHERE / df.filter()
Select	Choose/rename columns	SELECT / df.select()
Derived Column	Create calculated column	expression / withColumn()
Aggregate	Group and summarize	GROUP BY / groupBy().agg()
Conditional Split	Route rows by condition	CASE/filter branches
Join	Combine datasets	JOIN / df.join()

```
-- SQL example: aggregate sales
SELECT customer_id, SUM(amount) AS total_sales
FROM sales
GROUP BY customer_id;

# PySpark equivalent
from pyspark.sql import functions as F
result = (sales
    .groupBy("customer_id")
    .agg(F.sum("amount").alias("total_sales")))
```

When to use what — ADF Data Flow is convenient for visual/no-code transformations. Databricks/PySpark is often preferred when transformations become code-heavy, reusable, testable or very complex. The right choice depends on the project.

3.7 Triggers

A trigger starts a pipeline automatically. The transcript discusses scheduled/tumbling-window concepts and then builds a Storage Event trigger so a pipeline starts when a file arrives.

Trigger type	Starts pipeline when...	Typical use
Schedule	A clock schedule occurs	Daily/hourly batch
Tumbling Window	A fixed, stateful time window is due	Windowed batch processing/dependencies
Storage Event	A blob/file event occurs	Event-driven ingestion as soon as data lands

Course production scenario — A manager may drop a file at an unpredictable time—even 2:00 a.m. A Storage Event trigger removes the need to poll manually or wait for a fixed daily schedule.

3.8 Event Grid registration and troubleshooting

While configuring the storage-event trigger, the course hits a subscription/provider registration issue and registers Microsoft.EventGrid. This is a valuable real-world lesson: cloud pipelines fail for platform/configuration reasons as well as data/code reasons.

Troubleshooting habit — Read the exact error → identify the failing Azure capability → check subscription/provider registration, permissions, networking and resource configuration → retry → validate in Monitor.

3.9 Execute Pipeline and parent/child orchestration

Execute Pipeline lets one pipeline invoke another. This allows smaller pipelines to be composed into a parent/production workflow instead of building one enormous canvas.

Design	Advantage
Child pipelines	Encapsulate one responsibility
Parent pipeline	Coordinates the end-to-end sequence
On-success dependencies	Run next step only after prior success
Reusable pipelines	Same child can be invoked from multiple workflows

```
graph LR; SE[Storage Event] -- triggers --> PP[Parent Pipeline]; PP -- execute --> MC[Manager Copy]; PP -- execute --> GC[GitHub Copy]; MC -- on success --> MF[Metadata / ForEach]; MF -- files ready --> DF[Data Flow]; GC -- files ready --> DF; DF -- selected data --> RC[Reporting / Curated];
```

The final flow combines earlier skills: an event starts the parent pipeline; source data is copied; another pipeline can pull supporting data; metadata discovers files; ForEach and If Condition select the right files; parameterized Copy Activity moves them; Data Flow transforms data; and monitoring validates the run.

3.11 SQL and PySpark connection to the ADF pipeline

```
# Example notebook logic that ADF could orchestrate
from pyspark.sql import functions as F

raw = spark.read.option("header", True).csv("/mnt/raw/fact_sales/*.csv")
clean = (raw
        .withColumn("amount", F.col("amount").cast("double"))
        .filter(F.col("amount").isNotNull())
        .dropDuplicates())

(clean.write
 .format("delta")
 .mode("append")
 .save("/mnt/silver/fact_sales"))

-- Downstream SQL analytics
SELECT product_id,
       SUM(amount) AS revenue
FROM silver_fact_sales
GROUP BY product_id
ORDER BY revenue DESC;
```

3.12 Common mistakes

- Hard-coding filenames when the arrival pattern is dynamic.
- Confusing Linked Service (connection) with Dataset (specific data).
- Using a schedule trigger when the real requirement is “run immediately when a file arrives.”
- Forgetting to Publish All after authoring changes.
- Ignoring provider registration, RBAC or network errors because the pipeline logic “looks correct.”
- Building one huge pipeline instead of composing reusable child pipelines.
- Using ADF orchestration loops as if they were a substitute for distributed Spark transformations.

3.13 Interview drill

Interview: What is the difference between debug/run and publish?

Authoring/debug validates current design changes; publishing commits the authored factory changes so triggers and the live factory use them.

Interview: Why parameterize datasets?

To reuse one dataset definition for dynamic paths, filenames, tables or other values supplied at runtime.

Interview: When would you use Get Metadata + ForEach?

When a pipeline must discover files/folders dynamically and process each item rather than rely on a fixed list.

Interview: Schedule trigger vs Storage Event trigger?

Schedule is time-driven; Storage Event is event-driven and starts when a matching storage event occurs.

Interview: Why use parent/child pipelines?

Modularity, reuse, simpler debugging, clearer ownership of steps, and easier orchestration of complex workflows.

Quick revision

Remember	Meaning
Get Metadata	Read file/folder metadata such as childItems
ForEach	Loop over an array
If Condition	Branch on a boolean expression
Parameter	Runtime value replacing hard-coded configuration
Data Flow	Visual Spark-backed transformation
Storage Event Trigger	Start pipeline when storage event occurs
Execute Pipeline	Invoke a child pipeline
Monitor	Inspect pipeline/activity/trigger runs

Final Chapters 1–3 Cheat Sheet

Area	One-line answer
Cloud	On-demand computing resources delivered over a network
IaaS	Provider gives infrastructure; customer manages more
PaaS	Provider manages infrastructure/platform; customer focuses on data/app
SaaS	Finished software consumed as a service
ADLS Gen2	Azure Storage with hierarchical namespace for data-lake workloads
Cosmos DB	Managed NoSQL database family with multiple APIs/models
ADF	Data integration and orchestration service
Linked Service	Connection definition

Dataset	Specific data reference
Pipeline	Workflow
Activity	Action inside a pipeline
Get Metadata	Discover file/folder metadata
ForEach	Loop
If Condition	Branch
Parameter	Dynamic runtime input
Data Flow	Visual transformation
Trigger	Automatic pipeline starter
Execute Pipeline	Parent/child orchestration

Practice checklist

- Create a resource group and explain why it exists.
- Create a Storage Account and explain hierarchical namespace.
- Create an ADLS container/folder hierarchy for raw and curated data.
- Create ADF and connect it to ADLS.
- Build a Copy Activity pipeline from source to sink.
- Build Get Metadata → ForEach → If Condition → parameterized Copy Activity.
- Create a Storage Event trigger and verify it in Monitor.
- Create a parent pipeline that executes child pipelines.
- Rebuild the full flow from memory, then explain every object aloud as if answering an interview.

Study rule — If you can rebuild a pipeline but cannot explain Linked Service vs Dataset, source vs sink, metadata vs data, or schedule vs event trigger, you are memorizing clicks rather than learning the engineering concept.